

Python Packages on Ubuntu: System-Wide Installs and Virtual Environments (venv)

This guide covers two things: installing Python packages system-wide with pip3 on Ubuntu, and using venv (virtual environments) to install packages per-project instead. It also explains why venv is usually the better choice.

1. System-Wide Package Installation

Ubuntu ships with Python 3 as python3, and pip (the package installer) as pip3. Before installing packages, make sure both are installed:

```
sudo apt update
sudo apt install python3 python3-pip
```

Check the versions:

```
python3 --version
pip3 --version
```

To install a package system-wide (available to every user and script on the machine), use pip3 with sudo:

```
sudo pip3 install <package-name>
# example:
sudo pip3 install requests
```

On recent Ubuntu versions (23.04+), pip3 may refuse this with an "externally-managed-environment" error, because Ubuntu wants system Python packages managed by apt, not pip. You will see options such as:

- Install the apt version instead, e.g. `sudo apt install python3-requests`
- Force pip anyway (not recommended): `sudo pip3 install <package> --break-system-packages`
- Use a virtual environment instead (recommended — see Section 2)

To remove a system-wide package:

```
sudo pip3 uninstall <package-name>
```

To list what is currently installed system-wide:

```
pip3 list
```

2. What is venv?

venv is Python's built-in tool for creating a virtual environment: an isolated, self-contained folder with its own Python interpreter and its own set of installed packages, separate from the system-wide Python.

Instead of installing packages globally (affecting the whole machine), you install them only inside the venv, only for one project.

2.1 Creating a venv

Navigate to your project folder, then run:

```
cd my_project
python3 -m venv venv
```

This creates a folder named venv (you can name it anything) inside your project directory, containing a private copy of the Python interpreter and pip.

2.2 Activating a venv

Activation switches your terminal session to use the environment's Python and pip instead of the system ones:

```
source venv/bin/activate
```

Once activated, your terminal prompt will usually show the environment name in parentheses, e.g. (venv) user@machine:~\$. From this point on, plain python and pip commands (no "3" suffix needed) refer to the environment's own copies:

```
pip install requests
python my_script.py
```

Packages installed here stay inside the venv folder and do not affect the system Python or any other project.

2.3 Exiting (deactivating) a venv

To leave the environment and return to the system-wide Python:

```
deactivate
```

This is safe to run at any time and does not delete anything. Deleting the venv folder entirely (e.g. `rm -rf venv`) removes the environment and all packages installed in it.

3. System-Wide Install vs venv: Differences and Advantages

System-wide (sudo pip3)	venv (virtual environment)
Packages available to all users/projects	Packages available only inside that project
One version of a package for the whole machine	Each project can use a different version
Risk of version conflicts between projects	No conflicts — every project is isolated
Can break system tools that rely on apt-managed Python	System Python is untouched
Requires sudo / root privileges	No sudo needed
Hard to reproduce the exact setup elsewhere	Easy to reproduce via requirements.txt

Advantages of venv

- Isolation: different projects can require different, even conflicting, package versions without interfering with each other.
- No admin rights needed: everything installs into your own project folder, so no sudo is required.
- Safer: you cannot accidentally break system tools that depend on Ubuntu's own Python packages.
- Reproducibility: you can save the exact package list with `pip freeze > requirements.txt` and recreate the same environment anywhere with `pip install -r requirements.txt`.
- Easy cleanup: deleting the venv folder removes everything related to that project, leaving no trace on the system.

When system-wide installs make sense

- Command-line tools you want available everywhere on the machine, regardless of which project you're in.
- Simple scripts or one-off utilities where isolation doesn't matter.

For almost all application and project development, venv (or a similar tool) is the recommended approach.